



Algoritmos e Programação 1

Professora Ana Paula
Ciências da Computação
UFJ

Estruturas de dados homogêneas – Vetores



O que trataremos agora?

- **Vetores;**
- Fluxograma;
- Teste de mesa;



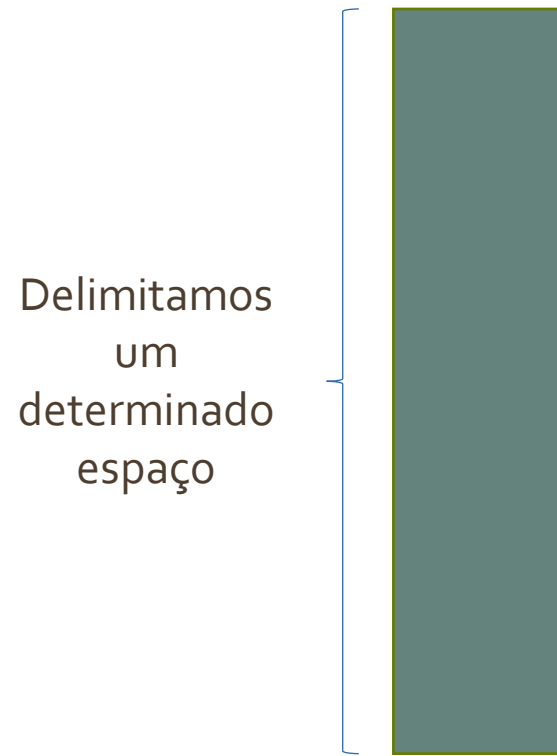
Problema de moradia em grandes centros: **apartamentos**;

Analogia com apartamentos, colocaríamos todos os **dados num único local (variável) - vetor**;

Primeiramente, iremos pensar como um apartamento com **um apartamento por andar** – vetor ou matriz unidimensional;

Introdução

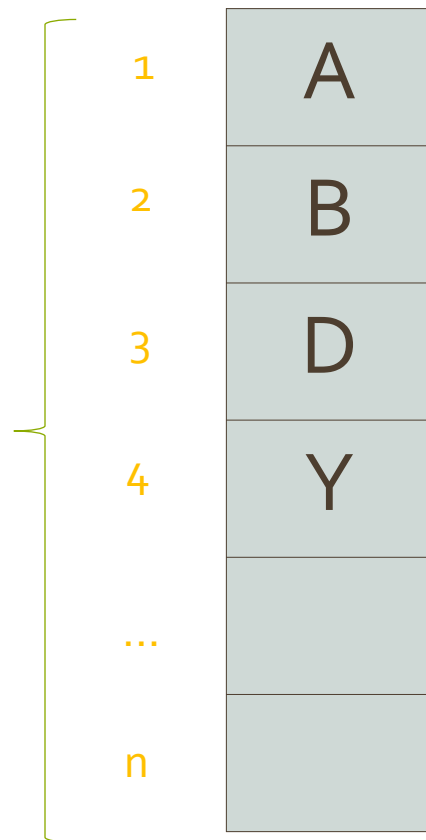
Introdução



Vetor é como se fosse uma caixa com vários lugares separados para guardar seus documentos, ou seja, uma variável onde se pode armazenar mais de um dado do mesmo tipo

Introdução

Dica: pensar
numa tabela do
MS Excel®



A diagram illustrating an array structure. On the left, a large curly bracket spans the rows. To the right of the bracket, indices are listed: 1, 2, 3, 4, ..., n. To the right of the indices is a vertical column of cells. The first four cells contain the letters A, B, D, and Y respectively. The fifth cell contains an ellipsis (...), and the sixth cell contains the letter n. The cells are light blue with black borders.

1	A
2	B
3	D
4	Y
...	
n	

Coleção de dados primitivos;

Os dados são do mesmo tipo

Índices para acessar as variáveis;

Estruturas de dados homogêneas – Vetores – *Array* – matriz 1D

Definição

Vetores ou estrutura de dados homogênea unidimensional. Isso quer dizer que se trata de um **conjunto de variáveis de mesmo tipo**, que possuem o **mesmo identificador (nome)** e são **alocadas sequencialmente na memória**. Como as variáveis têm o mesmo nome, **o que distingue é um índice que referencia sua localização dentro da estrutura**.

Estruturas de dados homogêneas – Vetores – *Array* – matriz 1D

Sintaxe de declaração VisuAlg

<nome>: **vetor** [<início>..<>fim>] **de** <tipo de dado>

Onde:

<nome> = nome da variável vetor;

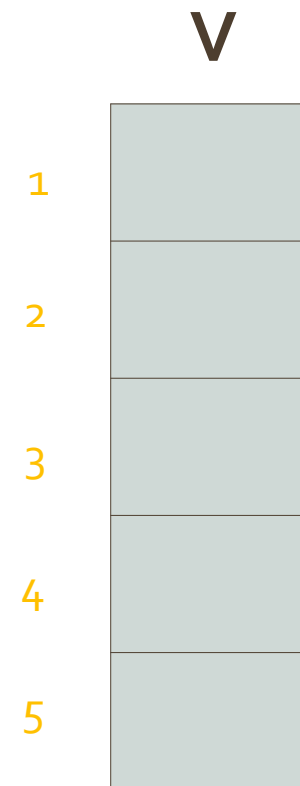
<início> = índice do primeiro elemento;

<fim> = índice do último elemento;

<tipo de dado> = tipo de dados dos elementos;

Var

V: **vetor** [1..5] **de** inteiro



Estruturas de dados homogêneas – Vetores – *Array* – matriz 1D

Sintaxe de declaração Linguagem C

```
<tipo> nome [NúmeroDeElementos];
```

Onde:

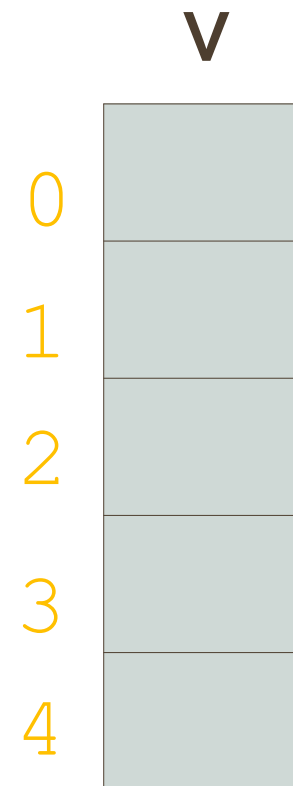
<tipo> = Tipo de dados – int, float, double;

<nome> = Nome da variável vetor;

<NúmeroDeElementos> = quantidade de elementos do vetor;

Exemplo:

```
int V[5];
```



Estruturas de dados homogêneas – Vetores – *Array* – matriz 1D

v	
0	45
1	0
2	-83
3	101
4	0

```
int v[5];
```

Atribuição

```
v[0] = 45;  
v[1] = 0;  
v[2] = -83;  
v[3] = 101;  
v[4] = v[1];
```

Estruturas de dados homogêneas – Vetores – *Array* – matriz 1D

<https://developerslife.tech/pt/>



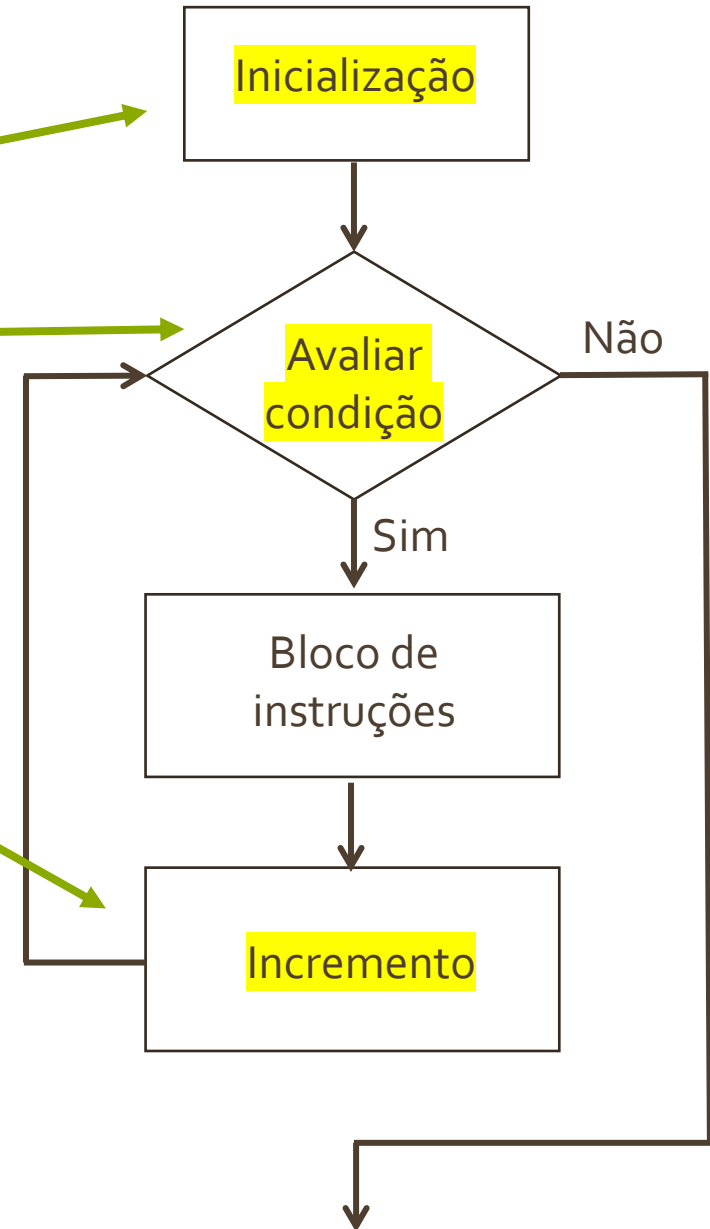
O que trataremos agora?

- Vetores;
- **Fluxograma;**
- Teste de mesa;

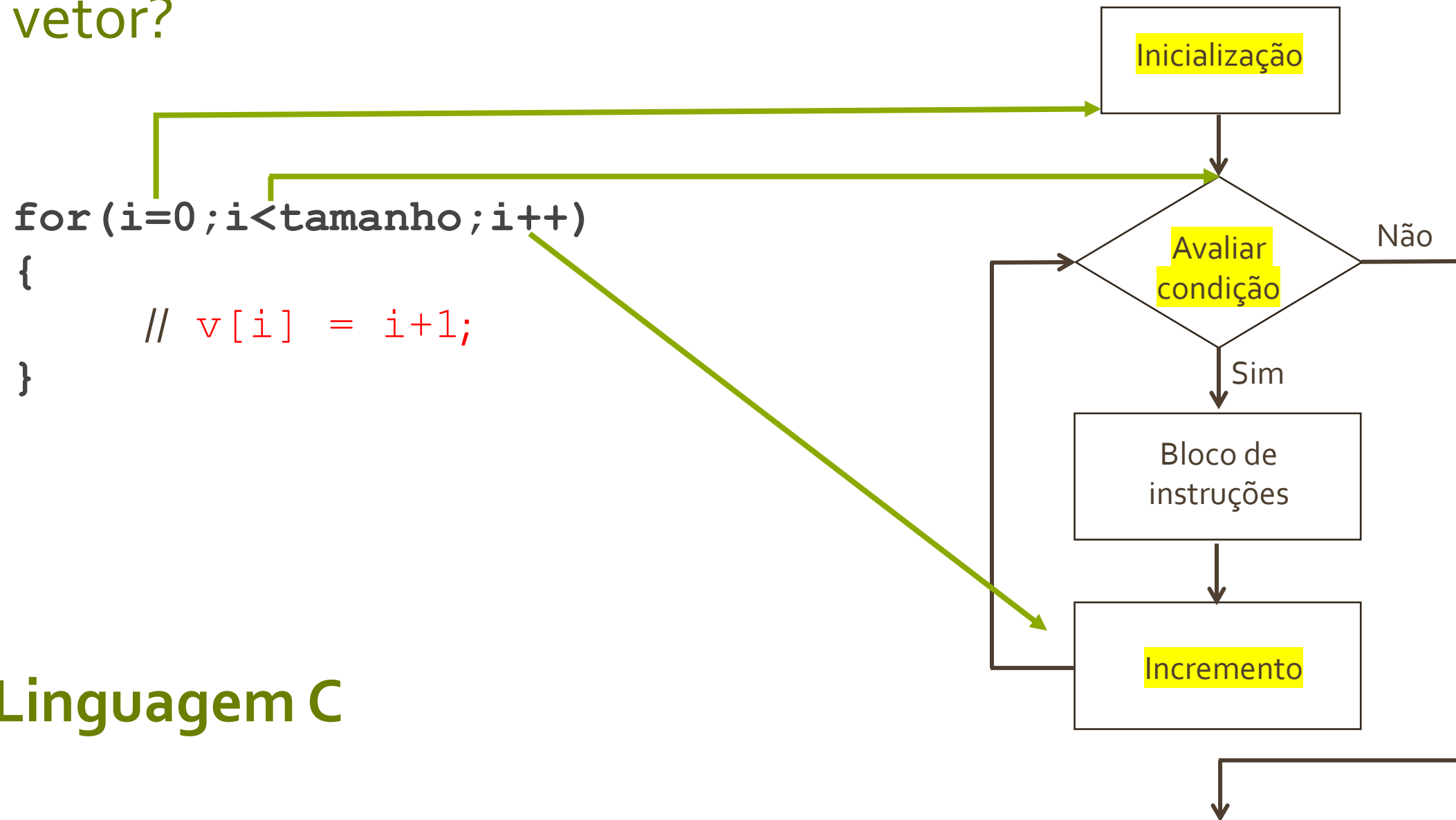
Como varrer automaticamente o vetor?

para <índice> de <valor inicial> ate <valor limite> [*<passo para alcance da condição>*] faça
 variável [índice]
fimpara

Pseudocódigo



Como varrer automaticamente o vetor?



Linguagem C

O que trataremos agora?

- Vetores;
- Fluxograma;
- **Teste de mesa;**

Estruturas de dados homogêneas

– Vetores – *Array* – matriz 1D

Teste de mesa realizado
em sala de aula

```
for (i=0;i<10;i++)  
{  
    v[i] = i+1;  
}
```

Problema de **exibir** os números de **1** a **10**.

```
int main ( )  
{ int v[10],i;
```

```
    for (i=0;i<10;i++)  
    {  
        v[i] = i+1;//scanf...  
    }
```

} montagem

```
    for (i=0;i<10;i++)  
    {  
        printf ("%d\n",v[i]);  
    }
```

} exibição

Estruturas de dados homogêneas

– Vetores – *Array* – matriz 1D

```
para <variável> de <valor inicial> ate <valor limite>  
[<passo para alcance da condição>] faça  
    Comando variável [n]  
fimpara
```

Problema de **exibir os números de 1 a 10.**

Algoritmo "semnome"

Var

V: vetor [1..10] de inteiro

i: inteiro

Inicio

para i de 1 ate 10 faça

v[i]<-i

fimpara

montagem

para i de 1 ate 10 faça

escreval(v[i])

fimpara

Fimalgoritmo

exibição

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10     escreval(v[i])
11   fimpara
12 Fimalgoritmo

```

[illegible]

Laço de repetição - para

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10     escreval(v[i])
11   fimpara
12 Fimalgoritmo

```

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10     escreval(v[i])
11   fimpara
12 Fimalgoritmo

```

[illegible]

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10     escreval(v[i])
11   fimpara
12 Fimalgoritmo

```

[illegible]

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10      escreval(v[i])
11    fimpara
12 Fimalgoritmo

```

[illegible]

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10     escreval(v[i])
11   fimpara
12 Fimalgoritmo

```

[illegible]

Laço de repetição - para

```
1 Algoritmo "semnome"
2 Var
3   V: vetor [1..3] de inteiro
4   i: inteiro
5 Inicio
6   para i de 1 ate 3 faca
7     v[i]<-i
8   fimpara
9   para i de 1 ate 3 faca
10    escreval(v[i])
11  fimpara
12 Fimalgoritmo
```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?

Laço de repetição - para

```
1 Algoritmo "semnome"
2 Var
3   V: vetor [1..3] de inteiro
4   i: inteiro
5 Inicio
6   para i de 1 ate 3 faca
7     v[i]<-i
8   fimpara
9   para i de 1 ate 3 faca
10     escreval(v[i])
11   fimpara
12 Fimalgoritmo
```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?

Laço de repetição - para

```
1 Algoritmo "semnome"
2 Var
3   V: vetor [1..3] de inteiro
4   i: inteiro
5 Inicio
6   para i de 1 ate 3 faca
7     v[i] <- i
8   fimpara
9   para i de 1 ate 3 faca
10    escreval(v[i])
11  fimpara
12 Fimalgoritmo
```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3

Laço de repetição - para

```
1 Algoritmo "semnome"
2 Var
3   V: vetor [1..3] de inteiro
4   i: inteiro
5 Inicio
6   para i de 1 ate 3 faca
7     v[i]<-i
8   fimpara
9   para i de 1 ate 3 faca
10    escreval(v[i])
11  fimpara
12 Fimalgoritmo
```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3

Laço de repetição - para

```
1 Algoritmo "semnome"
2 Var
3   V: vetor [1..3] de inteiro
4   i: inteiro
5 Inicio
6   para i de 1 ate 3 faca
7     v[i] <- i
8   fimpara
9   para i de 1 ate 3 faca
10    escreval(v[i])
11  fimpara
12 Fimalgoritmo
```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3

Laço de repetição - para

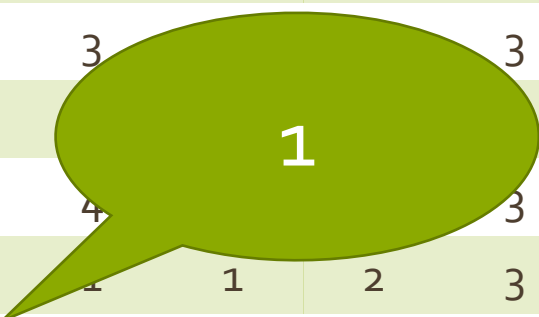
```
1 Algoritmo "semnome"
2 Var
3   V: vetor [1..3] de inteiro
4   i: inteiro
5 Inicio
6   para i de 1 ate 3 faca
7     v[i] <- i
8   fimpara
9   para i de 1 ate 3 faca
10    escreva(v[i])
11  fimpara
12 Fimalgoritmo
```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3
9	1	1	2	3

Laço de repetição - para

```
1 Algoritmo "semnome"
2 Var
3   V: vetor [1..3] de inteiro
4   i: inteiro
5 Inicio
6   para i de 1 ate 3 faca
7     v[i]<-i
8   fimpara
9   para i de 1 ate 3 faca
10    escreval(v[i])
11  fimpara
12 Fimalgoritmo
```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3
9	1	1	2	3
10	1	1	2	3



1

Laço de repetição - para

```
1 Algoritmo "semnome"
2 Var
3   V: vetor [1..3] de inteiro
4   i: inteiro
5 Inicio
6   para i de 1 ate 3 faca
7     v[i]<-i
8   fimpara
9   para i de 1 ate 3 faca
10    escreval(v[i])
11  fimpara
12 Fimalgoritmo
```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3
9	1	1	2	3
10	1	1	2	3
11	1	1	2	2

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10      escreval(v[i])
11    fimpara
12 Fimalgoritmo

```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3
9	1	1	2	3
10	1	1	2	3
11	1	1	2	3

[illegible]

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10     escreval(v[i])
11   fimpara
12 Fimalgoritmo

```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3
9	1	1	2	3
10	1	1	2	3
11	1	1	2	3

linha	i	V[1]	V[2]	V[3]
9	2	1	2	3
10	2	1	2	3

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10     escreval(v[i])
11   fimpara
12 Fimalgoritmo

```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3
9	1	1	2	3
10	1	1	2	3
11	1	1	2	3

[illegible]

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10      escreval(v[i])
11    fimpara
12 Fimalgoritmo

```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3
9	1	1	2	3
10	1	1	2	3
11	1	1	2	3

[illegible]

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10     escreval(v[i])
11   fimpara
12 Fimalgoritmo

```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3
9	1	1	2	3
10	1	1	2	3
11	1	1	2	3

linha	i	V[1]	V[2]	V[3]
9	2	1	2	3
10	2	1	2	3
11	2	1	2	3
9	3	1	2	3
10	3	1	2	3

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10     escreval(v[i])
11   fimpara
12 Fimalgoritmo

```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3
9	1	1	2	3
10	1	1	2	3
11	1	1	2	3

[illegible]

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i] <- i
8    fimpara
9    para i de 1 ate 3 faca
10      escreval(v[i])
11    fimpara
12 Fimalgoritmo

```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3
9	1	1	2	3
10	1	1	2	3
11	1	1	2	3

linha	i	V[1]	V[2]	V[3]
9	2	1	2	3
10	2	1	2	3
11	2	1	2	3
9	3	1	2	3
10	3	1	2	3
11	3	1	2	3
9	4	1	2	3

Laço de repetição - para

```

1  Algoritmo "semnome"
2  Var
3    V: vetor [1..3] de inteiro
4    i: inteiro
5  Inicio
6    para i de 1 ate 3 faca
7      v[i]<-i
8    fimpara
9    para i de 1 ate 3 faca
10     escreval(v[i])
11   fimpara
12 Fimalgoritmo

```

linha	i	V[1]	V[2]	V[3]
...	?	?	?	?
5	?	?	?	?
6	1	?	?	?
7	1	1	?	?
8	1	1	?	?
6	2	1	?	?
7	2	1	2	?
8	2	1	2	?
6	3	1	2	?
7	3	1	2	3
8	3	1	2	3
6	4	1	2	3
9	1	1	2	3
10	1	1	2	3
11	1	1	2	3

linha	i	V[1]	V[2]	V[3]
9	2	1	2	3
10	2	1	2	3
11	2	1	2	3
9	3	1	2	3
10	3	1	2	3
11	3	1	2	3
9	4	1	2	3
12	4	1	2	3

O que trataremos agora?

- Vetores;
- Fluxograma;
- Teste de mesa;